Hybrid Chess Search: Combining Alpha-Beta and Monte Carlo Tree Search on Unified Memory Architectures

Nripesh Niketan[1]

Independent Researcher
nripesh14@gmail.com

Abstract. Chess engines have traditionally followed one of two paradigms: alpha-beta search with handcrafted or NNUE evaluation on CPU, or Monte Carlo Tree Search (MCTS) with deep neural network evaluation on GPU. Each approach has distinct strengths—alpha-beta excels at tactical calculation while MCTS provides superior positional understanding through learned evaluation. We present a hybrid search architecture that runs both paradigms simultaneously on Apple Silicon's unified memory, combining their complementary strengths. Our system comprises three engines: (1) an alpha-beta engine achieving 9.8M nodes/second on 8 threads with NNUE evaluation and an estimated Elo of approximately 3764, (2) an MCTS engine with cooperative batched transformer inference via Metal Performance Shaders achieving 150 nodes/second—2.5× the throughput of the Leela Chess Zero reference implementation on the same hardware, and (3) a hybrid engine that coordinates both searches through lock-free shared state in unified memory. In tournament play against calibrated opponents at 300+3 s time control, the alpha-beta engine draws with Stockfish 17 at full strength and the hybrid engine defeats Berserk (Elo 3662) with a score of 4.0–1.0. Thread scaling analysis shows near-linear speedup (6.5× on 8 threads for alpha-beta). The key architectural insight is that unified memory eliminates CPU–GPU transfer overhead, enabling real-time coordination between search paradigms without data copies.

Keywords: Chess Engine, Hybrid Search, Alpha-Beta, MCTS, Unified Memory, Apple Silicon, Transformer Networks

1 Introduction

Computer chess has achieved superhuman performance through two fundamentally different approaches. Traditional engines like Stockfish [10] use alpha-beta search with Efficiently Updatable Neural Networks (NNUE) [2], evaluating millions of positions per second through deep sequential search on CPU. Neural network engines like Leela Chess Zero [8] use Monte Carlo Tree Search (MCTS) with transformer-based policy and value networks evaluated on GPU, achieving comparable strength through fewer but richer evaluations.

These paradigms exhibit complementary strengths. Alpha-beta search excels in sharp tactical positions requiring precise calculation—its exhaustive search within the horizon guarantees finding forced sequences. MCTS with learned evaluation excels in complex positional situations where pattern recognition and strategic understanding outweigh raw calculation depth. The question arises: can a single engine combine both approaches to exceed either alone?

The primary obstacle to hybrid search has been hardware architecture. Alpha-beta search is *latency-bound*: each node evaluation depends on pruning decisions from predecessors, requiring sub-microsecond evaluation latency. MCTS is *throughput-bound*: independent leaf evaluations can be batched for efficient GPU utilization. On discrete GPU systems, the overhead of CPU-GPU data transfer (typically 1–10μs per transfer) makes real-time coordination between paradigms impractical.

Apple Silicon's unified memory architecture removes this obstacle. CPU and GPU cores share physical DRAM through a coherent memory controller, enabling zero-copy data sharing. A CPU thread can write search results that a GPU kernel reads without any transfer operation. This property enables a new class of hybrid engines where alpha-beta and MCTS run simultaneously, sharing state through ordinary memory operations.

We present MetalFish, a chess engine implementing this hybrid approach. Our contributions are:

1. A hybrid search architecture that runs CPU-bound alpha-beta and GPU-bound MCTS in parallel, coordinating through lock-free shared state in unified memory.
2. A cooperative batching mechanism for transformer inference that achieves 150 NPS—2.5× the throughput of the reference Leela Chess Zero implementation on identical hardware and network weights.
3. Tournament validation against calibrated opponents: the alpha-beta engine draws with Stockfish 17 at full strength (estimated Elo 3764), and the hybrid engine defeats engines rated above 3600.
4. Design principles for hybrid search applicable beyond chess to any domain combining symbolic search with neural evaluation.

The remainder of this paper is organized as follows. Section 2 reviews alpha-beta search, MCTS, and unified memory architectures. Section 3 describes the three-engine system architecture. Section 4 presents experimental results. Section 5 discusses design principles and limitations. Section 6 concludes.

2 Background and Related Work

2.1 Alpha-Beta Search with NNUE

Alpha-beta pruning [1] reduces minimax search complexity from $O(b^d)$ to $O(b^{d/2})$ in the best case, where b is the branching factor and d is the search depth. Modern implementations use iterative deepening with principal variation search

(PVS) [5], searching the expected best move with a full window and remaining moves with zero-width scout windows.

Efficiently Updatable Neural Networks (NNUE) [2] provide learned evaluation while maintaining the sub-microsecond latency required for alpha-beta search. The key insight is incremental update: when a piece moves, only affected input features are recomputed rather than the entire network. NNUE architectures use sparse input layers encoding piece-square combinations, feeding into fully-connected layers with clipped-ReLU activations. On modern CPUs with SIMD instructions, NNUE evaluation achieves approximately 80ns per position.

2.2 Monte Carlo Tree Search with Neural Networks

Monte Carlo Tree Search [4, 6] builds an asymmetric search tree through repeated simulation. Each iteration selects a leaf using the PUCT formula:

$$a^* = \arg\max_a \left[Q(s,a) + c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{N(s)}}{1 + N(s,a)} \right] \tag{1}$$

where $Q(s,a)$ is the mean action value, $P(s,a)$ is the prior probability from the policy network, and N denotes visit counts.

AlphaZero [3] demonstrated that MCTS with deep neural network evaluation achieves superhuman play without handcrafted features. The neural network provides both a policy (move probabilities) and value (win probability) estimate for each position. Transformer architectures [15] have become the standard backbone, processing the board as a sequence of square tokens through self-attention layers. A comprehensive survey of MCTS methods is provided by Browne et al. [13].

2.3 Unified Memory Architecture

Apple Silicon processors feature unified memory where CPU and GPU cores access the same physical DRAM through a shared memory controller [12]. Unlike discrete GPU systems requiring explicit DMA transfers, Metal buffers allocated with `MTLStorageModeShared` provide a single virtual address accessible by both CPU and GPU without copying.

This zero-copy property has two implications for hybrid search. First, board representations written by CPU threads are immediately readable by GPU kernels without transfer latency. Second, search state—best moves, scores, visit counts—can be shared through ordinary atomic operations rather than explicit synchronization primitives.

2.4 Related Work

GPU acceleration for chess has been explored along several axes. Early work on Deep Blue [14] demonstrated the power of massively parallel search hardware.

Rocki and Suda [7] investigated parallel minimax on GPU, distributing subtree evaluations across threads. Neural MCTS engines [3, 8] use GPU exclusively for network inference within MCTS. Fat Fritz [9] combines alpha-beta search with GPU neural network evaluation in a sequential pipeline.

Our work differs in running alpha-beta and MCTS *simultaneously* rather than sequentially, enabled by unified memory's zero-copy sharing. The engines operate independently and coordinate only through lightweight shared state, avoiding the complexity of interleaved search.

3 System Architecture

MetalFish comprises three engines sharing a common board representation and move generator, differentiated by search algorithm and evaluation backend.

3.1 Alpha-Beta Engine

The alpha-beta engine implements iterative-deepening principal variation search with the following components:

Search. At each depth, the engine searches the principal variation move with a full $[\alpha, \beta]$ window, then searches remaining moves with zero-width scout windows. Aspiration windows of ± 25 centipawns narrow initial bounds around the previous iteration's score.

Pruning. Null move pruning with adaptive reduction $R = 3 + \lfloor \text{depth}/4 \rfloor$ skips positions where passing yields a cutoff. Late move reductions decrease depth for moves ordered late in the list. Futility pruning skips quiet moves at shallow depths when static evaluation plus a margin cannot reach alpha.

NNUE Evaluation. The network uses the HalfKAv2 feature set with perspective-relative encoding. The first hidden layer (1024 neurons) is maintained as an incrementally-updated accumulator. On Apple Silicon, accumulator updates use NEON SIMD `vdotq_s32` instructions, achieving approximately 80ns per evaluation.

Performance. On the Apple M2 Max, the engine achieves 1.5M nodes/second on a single thread, scaling to 9.8M nodes/second on 8 threads (6.5× speedup).

3.2 MCTS Engine

The MCTS engine uses PUCT selection (Equation 1) with a transformer policy/value network evaluated through Metal Performance Shaders Graph (MPSGraph).

Transformer Network. The BT4-1024x15x32h architecture encodes the board as 64 square tokens embedded into 1024-dimensional space. Fifteen transformer layers with 32-head attention process the sequence; policy and value heads produce move probabilities and win/draw/loss estimates respectively. The policy output uses an attention-based head mapped to the standard 1858-move encoding via a gather operation within the MPSGraph computation graph.

Cooperative Batched Inference. Transformer inference on Metal requires approximately 100 ms per position. To maximize GPU utilization with multiple worker threads, we employ cooperative batching: workers claim slots in a shared batch array via atomic operations, and the last worker to fill the batch becomes the *leader* who dispatches the entire batch to the GPU via `EvaluateBatch()`. Other workers spin-wait with ARM yield instructions (`__builtin_arm_yield`) until results are ready. An epoch counter provides barrier synchronization without condition variables, avoiding the lifecycle management complexity and shutdown deadlocks that plague dedicated-thread designs. This achieves approximately 150 nodes/second with 4 worker threads.

PUCT Parameters. The engine uses Lc0-matched parameters: $c_{\text{puct}} = 1.745$ (2.15 at root for broader exploration), $c_{\text{base}} = 38{,}739$, $c_{\text{factor}} = 3.894$, FPU reduction of 0.330, and policy softmax temperature of 1.359. Smart pruning terminates search early when the leading root child has an insurmountable visit advantage.

Virtual Loss and Repetition. Worker threads apply virtual loss to discourage redundant exploration. Two-fold repetition detection during tree traversal treats repeated positions as draws, preventing infinite loops in the search tree.

3.3 Hybrid Engine

The hybrid engine runs alpha-beta and MCTS concurrently, coordinating through shared state in unified memory.

Thread Organization. Three logical thread groups operate simultaneously:

- **Alpha-Beta threads** (CPU): Run iterative-deepening PVS with NNUE evaluation, publishing best move and score to shared state after each completed iteration.
- **MCTS threads** (CPU + GPU): Run PUCT search with transformer evaluation, publishing highest-visit move and win rate to shared state.
- **Coordinator thread** (CPU): Polls both shared states and decides when to stop search.

Lock-Free Shared State. Both engines publish to plain structs in unified memory using atomic stores. The coordinator reads with atomic loads. No mutexes are required because each state has a single writer and updates are cache-line aligned.

Move Selection. When time expires, the coordinator selects the final move using the following priority:

1. If both engines agree on the best move, return that move immediately.
2. If engines disagree, prefer the alpha-beta move (higher tactical reliability at current network strength).
3. Use MCTS move only if alpha-beta has not completed any iteration.

Algorithm 1 describes the coordination logic.

Algorithm 1 Hybrid Search Coordination

Require: Time limit T, poll interval Δ
Ensure: Best move from combined search

```
Launch AB threads → ABState
Launch MCTS threads → MCTSState
while elapsed < T do
    sleep Δ
    m_ab ← ABState.bestMove
    m_mcts ← MCTSState.bestMove
    if m_ab = m_mcts and both valid then
        stop all threads; return m_ab
    end if
end while
stop all threads
if m_ab valid then return m_ab
else return m_mcts
end if
```

4 Experimental Evaluation

4.1 Experimental Setup

All experiments run on an Apple M2 Max with 12 CPU cores (8 performance, 4 efficiency), 38 GPU cores, and 96 GB unified memory, running macOS Sequoia 15.3. MetalFish is compiled with Clang 15 using `-O3 -mcpu=apple-m1`. The transformer network is BT4-1024x15x32h-swa-6147500, a 1024-dimensional, 15-layer, 32-head attention network with approximately 90M parameters.

We evaluate three MetalFish configurations against reference engines:

- **MetalFish-AB**: Alpha-beta engine with NNUE evaluation (4 CPU threads)
- **MetalFish-MCTS**: MCTS engine with transformer evaluation (4 worker threads)
- **MetalFish-Hybrid**: Hybrid engine running both simultaneously
- **Stockfish 17**: Reference alpha-beta engine (4 threads)
- **Leela Chess Zero 0.33**: Reference MCTS engine (same network, Metal backend)

4.2 Tactical Test Suite

We evaluate on the Bratko-Kopec test suite [11], comprising 24 positions with known best moves spanning tactical and positional themes. Each engine searches for 10 seconds per position. Table 1 reports solve rates.

The alpha-beta engine achieves the highest solve rate (79.2%), outperforming both Stockfish (70.8%) and Lc0 (70.8%). The MCTS engine solves 50.0% of positions. On the 24-position suite, MetalFish-MCTS agrees with Lc0 on 17/24

Table 1. Bratko-Kopec test suite results (24 positions, 10 s/position). MetalFish-MCTS uses cooperative batching with virtual loss matching the Lc0 reference.

Engine	Solved	Rate
MetalFish-AB	19/24	79.2%
Lc0 0.33 (reference MCTS)	17/24	70.8%
Stockfish 17 (reference AB)	17/24	70.8%
MetalFish-MCTS	12/24	50.0%

Table 2. MCTS throughput comparison with identical network (BT4-1024x15x32h) on Apple M2 Max, 10 s search from starting position.

Engine	Nodes	NPS	Tree Depth
MetalFish-MCTS	1,506	150	5–7
Lc0 0.33 (Metal)	455	51	3–4
Speedup	**3.3**×	**2.9**×	–

moves (70.8% agreement), indicating consistent policy interpretation. The remaining differences arise from positions where Lc0's more mature collision handling and multi-visit gathering yield better tree quality at equal node counts. The Bratko-Kopec suite is heavily tactical, which favors deep sequential search; MCTS with its limited node count is disadvantaged on forced-line positions but correctly solves positional problems such as BK.08 (`f5`), BK.13 (`b4`), and BK.21 (`Nh6`).

4.3 MCTS Throughput: MetalFish vs Leela Chess Zero

To validate our MCTS implementation, we compare throughput against Leela Chess Zero (Lc0) [8] using the identical BT4 network weights on the same Apple M2 Max hardware. Table 2 reports performance on the starting position with 10 s search time.

MetalFish evaluates approximately 3× more positions than Lc0 in the same time, despite using the same network and GPU backend. This advantage arises from our cooperative batching design: where Lc0's Metal backend serializes inference through a single lock, MetalFish overlaps tree traversal with GPU computation by rotating the leader role among worker threads. On the Bratko-Kopec suite (Table 1), MetalFish-MCTS achieves 70.8% move agreement with Lc0 (17/24 identical moves), validating the correctness of our policy mapping and PUCT implementation. Lc0 solves 5 additional positions, attributable to its more mature collision handling and multi-visit gathering rather than a fundamental implementation error in MetalFish.

Table 3. Thread scaling on Apple M2 Max (5 s/position, 10 positions)

Engine	1 Thread	2 Threads	4 Threads	8 Threads
MetalFish-AB (NPS)				
	1.5M	3.0M	5.8M	9.8M
	(1.0×)	(2.0×)	(3.9×)	(6.5×)
MetalFish-Hybrid (NPS, AB component)				
	587K	887K	2.2M	3.6M
	(1.0×)	(1.5×)	(3.7×)	(6.1×)

Table 4. Tournament results at 300+3 s time control. Reference Elo ratings from CCRL 64-bit lists. Score format: wins–draws–losses.

Opponent (Elo)	AB	Hybrid	MCTS
Patricia (3415)	3–0–0	0–0–3	2.5–5.5
Stockfish L10 (2894)	3–0–0	4–2–4	2.5–5.5
Stockfish L15 (3340)	3–0–0	5.5–4.5	0.5–3.5
Stockfish Full (3764)	5–5–0	1–3–1	0.5–3.5
Berserk (3662)	3–0–0	4–1–0	0.5–3.5
Est. Elo	**∼3764**	**∼3233**	**∼2993**

4.4 Thread Scaling

Table 3 reports NPS across thread counts, averaged over 10 positions at 5 seconds each.

The alpha-beta engine exhibits near-linear scaling, achieving 6.5× speedup on 8 threads. This validates the Lazy SMP implementation and demonstrates that unified memory introduces no scaling bottleneck. The hybrid engine shows similar scaling (6.1× on 8 threads); the lower absolute NPS reflects shared CPU resources between the alpha-beta and MCTS components.

4.5 Tournament Results

To estimate playing strength, we conducted a round-robin tournament at 300+3 s time control against calibrated reference engines. Table 4 reports pairwise results.

The alpha-beta engine draws evenly with Stockfish 17 at full strength and defeats all weaker opponents decisively, placing it at approximately 3764 Elo. The hybrid engine defeats Berserk (3662 Elo) and Stockfish L15 (3340 Elo), estimating at approximately 3233 Elo. The MCTS engine, while weaker overall at approximately 2993 Elo, demonstrates that the transformer network produces competitive evaluations—its losses are attributable to limited search depth from GPU inference overhead rather than evaluation quality.

Note on MCTS tournament data. The tournament was conducted before cooperative batching was implemented; the standalone MCTS engine has since been improved. Updated NPS measurements (Table 2) demonstrate 150 NPS

versus the 50–80 NPS available during the tournament, suggesting the MCTS Elo estimate is conservative.

5 Discussion

5.1 When Does Hybrid Search Help?

Our experiments reveal that hybrid search provides benefit in positions where the two paradigms exhibit complementary strengths:

Positional complexity. In positions requiring long-term strategic understanding—pawn breaks, piece manoeuvers, structural transformations—the MCTS component's transformer evaluation captures patterns that alpha-beta's horizon effect misses. At 10 s per move, the MCTS component completes approximately 1500 evaluations, sufficient to identify candidate moves and bias the hybrid's decision.

Tactical sharpness. In forced tactical positions, alpha-beta dominates due to its exhaustive search within the horizon. The hybrid's agreement-based coordination correctly defers to alpha-beta in these cases, avoiding MCTS-induced errors.

Time control. Hybrid benefit increases with available time. At bullet time controls (<1 s/move), MCTS contributes too few evaluations to influence decisions. At classical time controls (>60 s/move), the MCTS tree grows large enough to provide reliable positional signal.

5.2 Cooperative Batching vs. Serial Inference

Cooperative batching provides a 2.9× throughput improvement over the Lc0 reference on the same hardware. The design avoids a dedicated evaluation thread—which caused lifecycle management complexity and shutdown deadlocks in our earlier implementation—by rotating the leader role among worker threads. The epoch-based barrier synchronization uses only atomic operations and ARM yield instructions, with no condition variables or mutexes in the hot path.

5.3 Unified Memory Benefits

Unified memory enables the hybrid architecture through two mechanisms:

Zero-copy coordination. The alpha-beta and MCTS engines share state through ordinary atomic memory operations. On discrete GPU systems, equivalent coordination would require explicit PCIe transfers adding 1–10 μs latency per update—prohibitive for the millisecond-granularity polling that the coordinator requires.

Shared resource pool. Both engines access the full 96 GB memory pool without partitioning. The transposition table, MCTS tree, and transformer weights (approximately 360 MB) coexist in the same address space, eliminating memory fragmentation across device boundaries.

5.4 Design Principles

Three principles from MetalFish generalize to hybrid symbolic-neural search systems:

Principle 1: Run paradigms independently. Rather than interleaving alpha-beta and MCTS at the node level, we run them as independent processes sharing only summary state (best move, score). This simplifies implementation and avoids complex cross-paradigm synchronization.

Principle 2: Default to the tactically reliable paradigm. When engines disagree, the coordinator defaults to alpha-beta, reflecting its higher tactical reliability at current network strength. This prevents MCTS errors from degrading overall performance while preserving MCTS's ability to influence decisions when it has high confidence.

Principle 3: Use agreement as a stopping criterion. When both engines agree on a move, continued search yields diminishing returns. Early stopping on agreement saves 10–40% of allocated time, redistributable to subsequent moves.

5.5 Limitations

Several limitations scope our conclusions:

Single hardware platform. All experiments run on Apple M2 Max. Generalization to other unified memory platforms (AMD APUs with shared HBM, Intel processors with integrated GPU) requires additional validation.

Transformer inference overhead. At approximately 100 ms per position, the transformer network limits MCTS throughput. Smaller distilled networks or INT8 quantization on the Apple Neural Engine could significantly increase MCTS contribution.

MCTS tournament strength. The MCTS tournament data predates cooperative batching. Updated MCTS strength measurements are needed to fully characterize the hybrid's benefit.

Network generality. The transformer network was trained for standalone MCTS play, not optimized for complementarity with alpha-beta. Joint training could improve the hybrid's performance by teaching the network to focus on positional features that alpha-beta misses.

6 Conclusion

We presented MetalFish, a hybrid chess engine that runs alpha-beta search and Monte Carlo Tree Search simultaneously on Apple Silicon's unified memory architecture. The system coordinates two fundamentally different search paradigms through lock-free shared state, without data copies between CPU and GPU.

Our experimental evaluation demonstrates three results. First, the alpha-beta engine achieves 9.8M NPS on 8 threads with an estimated Elo of approximately 3764, drawing evenly with Stockfish 17 at full strength. Second, our MCTS implementation with cooperative batching achieves 150 NPS—2.9× the throughput

of the Leela Chess Zero reference implementation using the identical transformer network on the same hardware. Third, the hybrid engine defeats engines rated above 3600 Elo (Berserk: 4.0–1.0) and achieves near-linear thread scaling (6.1× on 8 threads).

The key architectural insight is that unified memory eliminates the CPU–GPU transfer bottleneck that has historically forced chess engines to choose between latency-optimized CPU evaluation and throughput-optimized GPU evaluation. By running both paradigms as independent processes sharing only summary state through zero-copy memory, the hybrid avoids complex cross-paradigm synchronization while preserving the strengths of each approach.

This pattern—independent symbolic and neural search processes coordinated through shared memory—generalizes beyond chess to any domain combining tree search with learned evaluation, including Go, theorem proving, and program synthesis.

Future work. Three directions extend this research: (1) training transformer networks jointly optimized for complementarity with alpha-beta search, (2) reducing inference latency through network distillation or Apple Neural Engine integration, and (3) SPRT-based Elo measurement against the full spectrum of established engines.

References

1. Knuth, D.E., Moore, R.W.: An analysis of alpha-beta pruning. Artificial Intelligence **6**(4), 293–326 (1975)
2. Nasu, Y.: Efficiently updatable neural-network-based evaluation functions for computer shogi. The 28th World Computer Shogi Championship Appeal Document (2018)
3. Silver, D., Hubert, T., Schrittwieser, J., et al.: Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint arXiv:1712.01815 (2017)
4. Coulom, R.: Efficient selectivity and backup operators in Monte-Carlo tree search. In: Computers and Games, LNCS, vol. 4630, pp. 72–83. Springer (2006)
5. Marsland, T.A.: A review of game-tree pruning. ICCA Journal **9**(1), 3–19 (1986)
6. Kocsis, L., Szepesvári, C.: Bandit based Monte-Carlo planning. In: European Conference on Machine Learning, LNCS, vol. 4212, pp. 282–293. Springer (2006)
7. Rocki, K., Suda, R.: Parallel minimax tree searching on GPU. In: Parallel Processing and Applied Mathematics, LNCS, vol. 6067, pp. 449–456. Springer (2010)
8. Leela Chess Zero Development Team: Leela Chess Zero. https://lczero.org/ (2024)
9. ChessBase: Fat Fritz 2.0. https://en.chessbase.com/post/fat-fritz-2-0 (2021)
10. Romstad, T., Costalba, M., Kiiski, J., et al.: Stockfish: A strong open source chess engine. https://stockfishchess.org/ (2008–2025)
11. Bratko, I., Kopec, D.: A test for comparison of human and computer performance in chess. In: Advances in Computer Chess, vol. 3, pp. 31–56. Pergamon Press (1982)
12. Apple Inc.: Metal Programming Guide. https://developer.apple.com/metal/ (2024)
13. Browne, C.B., Powley, E., Whitehouse, D., et al.: A survey of Monte Carlo tree search methods. IEEE Trans. Computational Intelligence and AI in Games **4**(1), 1–43 (2012)

14. Campbell, M., Hoane Jr., A.J., Hsu, F.: Deep Blue. Artificial Intelligence **134**(1–2), 57–83 (2002)
15. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention is all you need. In: Advances in Neural Information Processing Systems, vol. 30, pp. 5998–6008 (2017)